SECUREVERSE

Blockchain-Anchored Decentralised Identity and End-to-End Encrypted Communication Platform

Deliverable 3 | Security Testing and Final Report

Field	Detail
Course	CY321 - Secure Software Development
Institution	Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Faculty	Faculty of Computer Science and Engineering
Authors	Rida Shahid Malik (2023610) | Isra Chaudhry (2023264)
Submission Date	05 May 2026
Report Version	Extended code document
Report Type	Security Testing, Static Analysis, Dynamic Testing, Code Review, extended explanations document

ABSTRACT

Centralised identity systems are the root architectural cause of the majority of large-scale data breaches, exposing billions of records annually by concentrating credential storage into permanently attractive targets. This report presents **SecureVerse**, a production-grade, full-stack Identity and Access Management platform that eliminates centralised credential storage by anchoring identities on Ethereum Sepolia using W3C Decentralised Identifiers (DIDs). Authentication is enforced through three cryptographically independent factors: DID ownership proven via secp256k1 challenge-response using the Ethereum personal_sign scheme, a Time-based One-Time Password (TOTP) second factor per RFC 6238, and an RS256-signed JSON Web Token issued with a 4096-bit RSA keypair using authlib 1.6.10. Messages are end-to-end encrypted in the browser using NaCl Box (X25519 Diffie-Hellman key exchange combined with XSalsa20-Poly1305 AEAD cipher), ensuring the relay server stores exclusively ciphertext it has no cryptographic capacity to decrypt. Audit logs employ HMAC-SHA256 chaining to provide tamper-evident forensic records, with critical event hashes additionally anchored on-chain via AuditLog.sol. Security testing produced the following outcomes: Pylint improved from 6.41/10 to 9.84/10; ESLint errors dropped from 27 to zero; Bandit SAST confirmed zero HIGH-severity findings; all 20 Postman API test cases passed; and two critical CVEs (CVE-2024-33664 and CVE-2024-33663) in the dependency python-jose were fully mitigated by migration to authlib 1.6.10. Seven secure code review findings were identified and resolved. Of 19 security requirements defined in Deliverable 1, 18 are fully met and one is partially met with a documented remediation path.

1. INTRODUCTION

The 2024 Verizon Data Breach Investigations Report documents over 8 billion records exposed in that calendar year alone. The root cause is architectural: centralised identity systems consolidate credential databases into single points of failure that are permanently and universally attractive to attackers. A single successful breach yields credentials valid across every service that trusts that central store.

Beyond breach risk, centralised identity creates a secondary problem: the identity provider observes all authentication and communication metadata. Every login, every message, every permission check is visible to the platform operator. This creates both surveillance exposure and a coercive attack surface where the provider itself may be compelled to expose user data.

SecureVerse was designed to address both problems at the architectural level rather than through incremental hardening of a fundamentally flawed model. Three design decisions drive the platform. First, identities are anchored on Ethereum Sepolia using W3C Decentralised Identifiers: there is no credential database to breach because the system stores no passwords, no password hashes, and no shared secrets that could be exfiltrated and cracked offline. Second, messages are end-to-end encrypted in the browser before transmission, so the relay server stores only ciphertext it cannot decrypt regardless of whether it is compromised. Third, audit logs are HMAC-SHA256 chained so any modification of any historical record is immediately detectable through chain verification.

This document covers the Deliverable 3 security testing phase. It is structured as follows: Section 2 provides a concise system architecture reference. Section 3 details the security methodology. Section 4 presents the threat model. Sections 5 and 6 cover static analysis results. Section 7 presents the dynamic testing suite with embedded

Postman evidence. Section 8 documents secure code review findings. Section 9 audits dependency currency and CVE exposure. Section 10 provides the full requirements traceability matrix. Section 11 outlines future work and Section 12 concludes.

2. SYSTEM ARCHITECTURE REFERENCE

2.1 FOUR-LAYER DESIGN

The platform is partitioned into four discrete architectural layers. Each layer has a bounded security responsibility and communicates with adjacent layers through authenticated, encrypted channels. This design ensures that compromise of any single layer does not automatically yield access to the others.

Layer	Components	Security Responsibility
Presentation	React 18.3.1, Vite 5.4.9, Tailwind CSS 3.4.14, Zustand 5.0.0, tweetnacl 1.0.3	Client-side NaCl Box encryption before transmission. Ethereum personal_sign challenge signing. PropTypes validation on all components. Zustand persisted auth store with 401 auto-logout.
Application	FastAPI 0.135.3, Python 3.12, asyncio throughout, slowapi 0.1.9, authlib 1.6.10	Three-factor authentication orchestration. Per-request RBAC enforcement via FastAPI dependency injection. Rate limiting at 10 req/min per IP on auth endpoints. RS256 JWT verification with Redis revocation check before decode.
Data	PostgreSQL 16 via asyncpg 0.31.0, Redis 7.4.0 (token blacklist, role cache, message queue)	Token revocation SADD set with O(1) SISMEMBER lookup. Role cache SETEX with 60-second TTL to prevent stale privilege. Encrypted message queue LPUSH/LRANGE with 24-hour TTL. Parameterised ORM queries exclusively via SQLAlchemy 2.0.49.
Blockchain	Ethereum Sepolia via Infura, Hardhat 2.22.0, Solidity 0.8.24, web3.py 7.15.0 (async)	Immutable DID ownership registry (DIDRegistry.sol). On-chain RBAC role assignment (RBACManager.sol). Audit event hash anchoring (AuditLog.sol). All RPC calls validated for HTTPS scheme at startup.

Table 1 - Four-layer architecture with security responsibilities per layer

2.2 SMART CONTRACT ARCHITECTURE

Three Solidity 0.8.24 contracts are deployed on the Ethereum Sepolia testnet. All were compiled with the optimiser enabled at 200 runs to minimise gas costs while preserving correctness. Deployment is managed via Hardhat 2.22.0 using a deterministic deployment script that saves deployed addresses to a JSON file for backend consumption.

- **DIDRegistry.sol:** stores the canonical mapping from DID string to Ethereum wallet address and from DID string to secp256k1 public key. Implements DID registration with uniqueness enforcement (a DID cannot be registered twice), public key rotation via updatePublicKey(), DID deactivation via deactivateDID(), and a two-step ownership transfer pattern to prevent accidental ownership loss. The onlyDIDOwner modifier verifies msg.sender against the stored owner address and checks the deactivated flag before any mutation.
- **RBACManager.sol:** stores role assignments in a mapping(address => string) with owner-only write access enforced via the onlyOwner modifier. Every assignRole() call emits a RoleAssigned(address indexed user, string role) event, creating an immutable on-chain audit trail of all role changes that cannot be modified even with database administrator access.
- **AuditLog.sol:** emits EventAnchored(bytes32 indexed eventHash, uint256 timestamp) events for critical security events. The contract deliberately stores no state, keeping gas costs minimal. The indexed hash enables efficient Ethereum log filtering. The backend calls anchor(bytes32) after login, logout, registration, and account lockout events, providing an external tamper-evidence anchor independent of the PostgreSQL database.

2.3 API ENDPOINT INVENTORY

Method	Endpoint	Auth Required	Minimum Role	Purpose
GET	/health	None	None	Liveness probe. Suppresses Server and X-Powered-By headers. Returns {status: ok, timestamp}.
POST	/auth/register	None	None	Register DID, wallet address, and public key. Generates TOTP secret, provisions QR code, anchors DID on-chain via DIDRegistry.registerDID(). Returns base64 QR code.
POST	/auth/login	None	None	Three-factor authentication: ecrecover challenge-response, TOTP verify, then RS256 JWT issue. Implements 5-attempt lockout with 15-minute penalty. Returns access_token and refresh_token.
POST	/auth/logout	JWT	User	Adds token JTI to Redis SADD revocation set. All subsequent SISMEMBER checks will block this token.
GET	/auth/me	JWT	User	Returns user DID, role, and registration timestamp. Validates token and revocation status.
POST	/messages/send	JWT	User	Accepts ciphertext_b64 and nonce_b64 only. LPUSH to Redis recipient queue with 24-hour EXPIRE. Server never receives or stores plaintext.

Method	Endpoint	Auth Required	Minimum Role	Purpose
GET	/messages/inbox	JWT	User	LRANGE from Redis queue. Returns array of {ciphertext_b64, nonce_b64, sender_did}. No plaintext field present.
GET	/audit/logs	JWT	Admin	Returns paginated HMAC-chained audit log. Supports filter by user_did and action_type query parameters.
GET	/audit/verify	JWT	Admin	Recomputes full HMAC chain from GENESIS entry. Returns {chain_valid: true/false, entry_count, verification_timestamp}.

Table 2 - API endpoint inventory with authentication and RBAC requirements

3. SECURITY METHODOLOGY

3.1 THREE-FACTOR AUTHENTICATION PROTOCOL

The login protocol is designed so that compromise of any single factor is insufficient to gain access. The three factors are cryptographically independent: the secp256k1 private key (possession), the TOTP secret bound to a physical authenticator device (possession, separate device), and knowledge of the DID string (identifier). An attacker who steals the JWT database gains nothing because JWTs are verified against the RS256 public key. An attacker who reads the TOTP secret from the database still cannot authenticate without the private key. An attacker who obtains the private key still requires the TOTP code.

Step	Factor	Mechanism	Failure Path	Audit Event
1	DID lookup	SELECT user WHERE did = $1 via SQLAlchemy	401 Unknown DID (constant-time response to prevent enumeration)	None
2	Account status	Check locked_until > now() in users table	429 Account locked with remaining_seconds	None
3	Key ownership	ecrecover(keccak256(prefix + challenge), signature) == DIDRegistry.getOwner(did)	401 Invalid signature; increment failed_attempts	None on failure unless lockout triggered
4	Lockout trigger	If failed_attempts >= 5 after step 3 failure	Set locked_until = now() + 15min; 429 returned	ACCOUNT_LOCKED event written to HMAC chain
5	TOTP code	pyotp.TOTP(secret).verify(code, valid_window=1) allowing 30s clock skew	401 Invalid TOTP (no attempt counter increment to prevent TOTP enumeration)	None
6	Token issue	authlib JsonWebToken with RS256, sub=DID, role=role, 15-min exp	200 access_token + refresh_token returned	USER_LOGIN written to HMAC chain + Sepolia anchor

Table 3 - Three-factor authentication protocol with failure paths and audit events

The challenge-response mechanism is constructed to prevent replay attacks. The client generates a 32-byte cryptographically random challenge on every login attempt using the browser Web Crypto API. The challenge is signed using the Ethereum personal_sign scheme: the message is prefixed with the string \x19Ethereum Signed

Message:\n32 before keccak256 hashing, matching the EIP-191 standard. The backend recovers the signer address using eth_account.recover_message() with encode_defunct() and compares it against the on-chain registered owner. The private key never leaves the client browser in any form.

3.2 JWT TOKEN MANAGEMENT AND CVE MITIGATIONS

The original project specification listed python-jose as the JWT library. During the Deliverable 3 security review, python-jose was identified as affected by two critical CVEs. The decision was made to remove it entirely and replace it with authlib 1.6.10, which has no known CVEs and is actively maintained by a dedicated team.

CVE	CVSS Severity	Vulnerability Class	Impact on python-jose	SecureVerse Mitigation
CVE-2024-33664	HIGH	JWT algorithm confusion: the library accepts tokens with alg header set to none, bypassing signature verification entirely	An attacker who intercepts or fabricates a token can set alg:none and the library will accept it without any signature check, allowing arbitrary claims	authlib enforces RS256 exclusively. Any token presenting a different algorithm value raises an immediate exception before any claim is processed. There is no code path that accepts alternative algorithms.
CVE-2024-33663	HIGH	RSA key confusion: the library can be tricked into using the RS256 public key as an HMAC-SHA256 secret when an attacker submits a token with alg:HS256	An attacker who knows the RS256 public key (which is public) can sign arbitrary tokens using it as an HMAC secret and they will be accepted	RS256-only enforcement in authlib means no HS256 or HS384 algorithm is ever accepted. The public key is never used in a symmetric context.

Table 4 - CVE mitigations applied by replacing python-jose with authlib 1.6.10

Token lifecycle in the implemented system: access tokens expire after 15 minutes from issuance. Refresh tokens expire after 7 days. On any logout call, the token's JTI (JWT ID claim) is written to a Redis SADD set with a TTL matching the token's remaining validity. Every subsequent request verification calls Redis SISMEMBER before any decoding occurs. This O(1) operation ensures revoked tokens are blocked without additional database queries. The role claim embedded in the JWT is treated as a hint only: it is re-validated against PostgreSQL on every request to a protected endpoint, preventing stale privilege escalation in the event of a role change.

3.3 END-TO-END ENCRYPTED MESSAGING ARCHITECTURE

The E2EE model uses NaCl Box, which combines X25519 elliptic-curve Diffie-Hellman for key agreement with XSalsa20-Poly1305 for authenticated encryption. Key agreement is ephemeral per message: the sender generates a fresh X25519 keypair for each message, uses it to derive a shared secret with the recipient's long-term public key, and then discards the ephemeral private key. This provides forward secrecy: historical messages remain confidential even if a long-term identity key is later compromised.

Phase	Location	Operation	What Is Transmitted
Key Discovery	Client browser	Call DIDRegistry.getPublicKey(did) via Infura RPC to retrieve recipient's secp256k1 public key from on-chain storage	Nothing transmitted to application server; query goes directly to Ethereum node
Encryption	Client browser (sandboxed)	nacl.box(message, nonce, recipientPublicKey, senderEphemeralSecretKey) using tweetnacl 1.0.3	Operation is local; no plaintext ever transmitted
Transmission	Client to application server	POST /messages/send with body containing ciphertext_b64 (base64 NaCl Box output) and nonce_b64 (24-byte random nonce)	Only ciphertext and nonce; server never receives plaintext or any private key
Storage	Application server to Redis	LPUSH msgs:{recipient_did} with JSON payload {ciphertext_b64, nonce_b64, sender_did}; EXPIRE set to 86400 seconds	Redis stores only ciphertext and nonce
Retrieval	Application server to client	GET /messages/inbox triggers LRANGE msgs:{did}; response body contains array of {ciphertext_b64, nonce_b64, sender_did}	Only ciphertext and nonce returned to client
Decryption	Recipient browser (sandboxed)	nacl.box.open(ciphertext, nonce, senderPublicKey, recipientPrivateKey); Poly1305 MAC authenticates before decryption	Plaintext exists only in recipient browser memory; never stored or transmitted

Table 5 - E2EE message lifecycle: server holds only ciphertext at every phase

3.4 HMAC-SHA256 AUDIT CHAIN

The audit log implements a forward-linked hash chain using HMAC-SHA256. Each entry's authentication code is computed over the concatenation of the previous entry's HMAC, the user DID, the action string, and the ISO-8601 timestamp. The genesis entry uses the literal string GENESIS as the predecessor HMAC. This means that modifying any entry changes its HMAC, which then invalidates every subsequent entry in the chain because each entry incorporates its predecessor's HMAC as an input. The chain cannot be silently repaired without recomputing all subsequent HMACs with the correct secret key.

The HMAC secret key is loaded exclusively from the SECRET_KEY environment variable, which is also the JWT signing secret. It is never stored in source code or version control. Critical events (login, logout, registration, account lockout) are additionally submitted to AuditLog.sol on Sepolia, which emits an immutable on-chain event. Even with full database write access, an attacker cannot modify these on-chain anchors.

4. THREAT MODEL

Three attacker archetypes were modelled. A network-level attacker can intercept, modify, or replay traffic in transit. A compromised-server attacker has full read and write access to the PostgreSQL database and Redis instance but does not hold the application's private keys. An authentication-level attacker has obtained one or more partial credentials such as a stolen JWT, a leaked TOTP secret, or knowledge of a DID string, and is attempting to use them to gain unauthorised access.

Attack Vector	Attacker Category	Mitigation Implemented	Residual Risk Assessment
JWT algorithm confusion (alg:none)	Auth-level	authlib 1.6.10 enforces RS256 exclusively; exception raised on any other algorithm value including none, HS256, HS384, RS384	Negligible. No code path processes a token with a non-RS256 algorithm. CVE-2024-33664 fully mitigated.
Token replay after logout	Auth-level	Redis SADD blacklist with per-token TTL. SISMEMBER check on every request before JWT decode begins.	Negligible. O(1) lookup cost means no performance-based bypass is viable. Tokens expire naturally within 15 minutes.
Brute-force credential attack	Auth-level	5-consecutive-failure account lockout (15 minutes). Rate limiting at 10 requests/minute per IP via slowapi.	Low. Distributed attacks from multiple IPs can partially bypass per-IP rate limiting. TOTP as second factor remains a strong barrier.
TOTP secret theft from database	Compromised server	TOTP secret is stored encrypted in PostgreSQL. Even with DB read access, the attacker also requires the private key to complete authentication.	Low. Requires simultaneous compromise of database and user's private key.
Database breach exposing messages	Compromised server	Only ciphertext_b64 and nonce_b64 stored. Server has no private key. Decryption requires the recipient's private key held only in the browser.	Negligible. Attacker gains useless ciphertext with no means of decryption.
Database breach exposing credentials	Compromised server	No passwords or password hashes stored. Authentication requires a secp256k1 private key never transmitted to the server.	Low. Attacker still requires the user's private key to authenticate. Only the DID string and TOTP secret are stored.

Attack Vector	Attacker Category	Mitigation Implemented	Residual Risk Assessment
Audit log tampering	Compromised server	HMAC-SHA256 chain detects any modification. Critical events anchored on Sepolia blockchain cannot be modified even with DB write access.	Low. Chain tampering is detectable via GET /audit/verify. On-chain anchors provide independent verification.
SQL injection	Network-level	SQLAlchemy ORM with parameterised queries throughout all database operations. No raw SQL strings exist in the codebase.	Negligible. Confirmed by code review. Bandit B608 (raw SQL) produced zero findings.
Role privilege escalation via stale JWT claim	Auth-level	Role claim in JWT is re-validated from PostgreSQL on every protected request. Redis role cache expires within 60 seconds.	Low. Maximum stale privilege window is 60 seconds, bounded by the Redis TTL.
Private key exfiltration via XSS	Network-level	Private key entered in browser input for login. No CSP headers currently implemented. Future: MetaMask integration.	Medium. Browser XSS could expose the private key. Highest-priority future work item.
DID impersonation via forged challenge response	Auth-level	ecrecover verifies the secp256k1 signature and compares recovered address against the on-chain DIDRegistry.getOwner(did) value.	Negligible. Requires breaking secp256k1, which is computationally infeasible.
Server version and technology disclosure	Network-level	Server and X-Powered-By headers overwritten in FastAPI middleware. Error responses follow RFC 7807 format without stack traces.	Low. Postman test TC-19 confirmed no version disclosure in production-equivalent configuration.

Table 6 - Threat model attack resistance matrix covering 12 attack vectors

5. STATIC ANALYSIS - PYTHON BACKEND (PYLINT)

Pylint was run against the full backend package comprising 13 modules and 251 statements. The tool was configured with maximum-line-length set to 100 characters and R0903 (too-few-public-methods) disabled for small Pydantic data classes. The scan was executed before any code review fixes were applied to establish a baseline, then again after all findings were resolved.

Metric	Before Fixes	After Fixes	Delta	Primary Remediation Applied
Overall Score	6.41 / 10	9.84 / 10	+3.43	Systematic documentation, import hygiene, and exception specificity
Convention (C) violations	80	3	-77	Docstrings added to all 13 modules and all public functions. Import order corrected. Naming conventions enforced throughout.
Warning (W) violations	7	0	-7	Broad except clauses (W0718/CWE-396) replaced with specific exception types. Unused function arguments prefixed with underscore.
Refactor (R) suggestions	3	1	-2	Duplicate validation logic extracted into shared helper. One remaining item is a SQLAlchemy ORM false positive (R0801 duplicate code across async session context managers).
Error (E) findings	0	0	0	No errors found in either scan. Indicates no missing imports, undefined variables, or type errors.
Modules with docstrings	23% (3/13)	100% (13/13)	+77%	Module-level docstrings added to all 13 files describing purpose, security responsibility, and key functions.
Functions with docstrings	31%	100%	+69%	Args, Returns, and Raises documented in Google style for all public functions including security-critical auth and RBAC functions.

Table 7 - Pylint results before and after code review fixes (13 modules, 251 statements)

5.1 SECURITY-RELEVANT PYLINT FINDINGS

The most security-relevant finding in the Pylint scan was W0718 (broad-exception-caught, CWE-396: Declaration of Catch for Generic Exception) appearing in three locations in the auth router. Catching bare Exception is a security concern because it can mask cryptographic verification failures, database errors, and other security-relevant exceptions by converting them into generic HTTP 500 responses that do not trigger appropriate logging or alerting. The fix involved replacing each broad except clause with the specific exception types expected at that point in the control flow.

The second notable finding was W0612 (unused-variable) in a database iteration loop where a loop variable named result was assigned but never used, suggesting either dead code or an incomplete implementation. Review confirmed the loop was intentionally iterating for its side effects. The variable was renamed to underscore to make the intent explicit and the warning was resolved.

The three remaining Refactor suggestions after fixing were all R0801 (duplicate-code) false positives triggered by the repeated pattern of SQLAlchemy async session context managers, which are structurally identical by design. Pylint's duplicate-code detector does not account for this pattern. The one remaining warning after fixing is this SQLAlchemy false positive, which was accepted with a documented suppression comment.

6. STATIC ANALYSIS - FRONTEND AND SAST

6.1 ESLINT - REACT FRONTEND

ESLint was configured with eslint-plugin-react 7.37.5 for React-specific rules, eslint-plugin-security 4.0.0 for security-relevant JavaScript patterns, and typescript-eslint 8.59.1 for type-aware linting. The initial scan found 27 errors across 8 of the 19 source files. After applying all fixes, ESLint produced a clean exit with 0 errors and 0 warnings.

ESLint Rule Violated	Occurrences	Affected Files	Security and Quality Impact	Fix Applied
react/prop-types	22	MessageCard, AuditRow, Layout, ProtectedRoute, QRModal, BubbleBackground	Missing runtime type validation on component props means unexpected data types from API responses propagate silently through the component tree, potentially rendering incorrect security status information or allowing prototype pollution via malformed response objects	Complete PropTypes definitions added to all six components. PropTypes.isRequired applied to all security-relevant props including token, did, and role. PropTypes.shape() used for structured props such as msg and entry.
no-empty	2	Navbar.jsx (logout catch block), useAuth.js (refresh catch block)	Silent catch blocks (CWE-390: Detection of Error Condition Without Action) hide runtime errors including authentication failures and network errors. A failed logout call would be invisible to monitoring and the session state might become inconsistent.	console.error logging added to both catch blocks with contextual messages. The logout flow still clears local state regardless of API success to ensure the user's session is always terminated client-side.
no-unused-vars	2	LoginPage.jsx, RegisterPage.jsx	Dead imports increase bundle size and create potential naming conflicts that could shadow security-relevant identifiers in future development	Dead imports removed. ESLint no-unused-vars now enforces this constraint on all future changes.
react-in-jsx-scope	1	One utility component	Missing React import causes runtime failures in specific bundler configurations, particularly when deploying to environments that do not	React import added to the affected file.

ESLint Rule Violated	Occurrences	Affected Files	Security and Quality Impact	Fix Applied
			support the newer JSX transform	

Table 8 - ESLint findings with security impact and remediation (27 errors reduced to 0)

6.2 BANDIT - PYTHON SAST

Bandit version 1.7.x was executed with recursive scanning (-r), low severity threshold (-l) to capture all findings including informational items, and JSON output format for systematic review. The scan covered all 13 production modules. No findings changed between the pre-fix and post-fix scans because all Bandit findings were configuration-level concerns rather than code logic errors.

Bandit ID	File	Severity	Confidence	Description and Technical Analysis	Disposition
B105	core/config.py	MEDIUM	LOW	Hardcoded password detection: Bandit flagged the variable name SECRET_KEY as potentially containing a hardcoded credential. This is a static name-pattern match, not detection of an actual value.	Accepted. The field is declared as a required pydantic-settings BaseSettings field with no default value. The application raises a validation error at startup if SECRET_KEY is absent from the environment. No fallback or default string exists. This is the recommended pattern per OWASP Secrets Management Cheat Sheet. Suppression comment added with justification.
B310	blockchain/did.py	MEDIUM	MEDIUM	urllib.request.urlopen with variable URL: Bandit flagged the Infura RPC endpoint construction because the URL is assembled from an environment variable rather than being a literal string.	Accepted. The URL is constructed as https://sepolia.infura.io/v3/{INFURA_PROJECT_ID}. A startup validation function was added (CR-05 from code review) that raises ValueError if the URL does not begin with https://. Non-HTTPS URLs are now rejected at startup, not at request time. The INFURA_PROJECT_ID variable contains only a project identifier, not a URL, so there is no open-redirect risk.
B311	auth/tokens.py	LOW	HIGH	Standard pseudo-random generator: Bandit flagged a call that generates	False positive. The flagged function is secrets.token_hex() which uses the operating system CSPRNG via /dev/urandom on Linux. This is confirmed in the CPython source. Bandit B311 is

Bandit ID	File	Severity	Confidence	Description and Technical Analysis	Disposition
				random values, incorrectly classifying Python's secrets module as a pseudo-random generator.	documented to produce false positives on the secrets module. The suppression was added with a comment citing the CPython source reference.
B101	tests/test_auth.py, tests/test_rbac.py, tests/test_audit.py	LOW	HIGH	Assert statement usage in test files.	Accepted. Assert statements are the standard pytest assertion mechanism and are expected and required in test code. Bandit B101 is explicitly documented as applying only to production code. The findings are in files under the tests/ directory. No suppression was added because this is expected pytest usage.

Table 9 - Bandit SAST findings with full technical disposition (0 HIGH severity findings)

The absence of HIGH-severity Bandit findings is particularly meaningful given the cryptographic nature of the codebase. Bandit's high-severity rules cover SQL injection (B608), use of weak cryptographic primitives (B304, B305, B306), shell injection (B602, B603), and hardcoded passwords in source (B106). None of these fired, confirming that the implementation avoids the most common categories of cryptographic and injection vulnerabilities detectable through static analysis.

7. DYNAMIC TESTING - POSTMAN API TEST SUITE

Twenty test cases were designed to achieve full coverage of all nine API endpoints, all security controls, all error conditions, and the two critical CVE mitigations. Tests were executed against a locally running FastAPI instance (uvicorn on port 8000) with the complete stack: PostgreSQL 16, Redis 7.4.0, and a Hardhat local blockchain node simulating Sepolia contract interactions. Each test specifies an expected HTTP status code, a response body assertion, and a documented security rationale.

The test suite is structured into five categories: liveness and basic functionality (TC-01 to TC-02), authentication security (TC-03 to TC-09), authorisation and RBAC (TC-10 to TC-13), end-to-end encrypted messaging (TC-14 to TC-15), and audit log integrity (TC-16 to TC-20). All 20 test cases passed on the final submission build.

ID	Endpoint	Test Scenario	Method	Expected Status	Security Rationale
TC-01	/health	Server liveness probe and header inspection	GET	200 OK	Confirms service availability. Response headers must not expose Server version or X-Powered-By. Validated via TC-19.
TC-02	/auth/register	Register valid DID with correct secp256k1 keypair	POST	200 + QR code	Validates the complete registration flow including on-chain DID anchoring and TOTP provisioning. QR code base64 in response.
TC-03	/auth/login	Login attempt with unregistered DID	POST	401	System rejects unknown identities with a constant-time response to prevent DID enumeration. Response message must not indicate whether the DID exists or not.
TC-04	/auth/login	Login with valid DID and signature but incorrect TOTP code	POST	401	Confirms second factor is independently enforced. A valid secp256k1 signature is insufficient without the correct TOTP code.
TC-05	/auth/me	Submit JWT token with alg header set to none (CVE-2024-33664)	GET	401	Directly validates the CVE-2024-33664 mitigation. authlib must reject algorithm confusion attacks without processing any claim.

ID	Endpoint	Test Scenario	Method	Expected Status	Security Rationale
TC-06	/auth/me	Submit valid token after calling /auth/logout (token replay)	GET	401	Validates the Redis revocation blacklist. A token that was valid before logout must be rejected after it has been blacklisted.
TC-07	/auth/login	Exceed rate limit: 11 sequential login requests from same IP	POST x11	429 on 11th	Validates slowapi rate limiter at 10 req/min. The 11th request within 60 seconds must return 429 Too Many Requests.
TC-08	/auth/login	5 consecutive logins with valid DID but invalid signature	POST x5	429 on 5th	Validates account lockout at 5 failed attempts. The 5th failed attempt triggers a 15-minute lockout and writes ACCOUNT_LOCKED to the audit chain.
TC-09	/auth/me	GET request with no Authorization header	GET	401	HTTPBearer dependency must reject requests lacking credentials without revealing any system information.
TC-10	/audit/logs	GET audit log with valid JWT of role=user (not admin)	GET	401 or 403	RBAC enforces admin-only access to audit endpoints. User-role tokens must be rejected regardless of their validity.
TC-11	/messages/inbox	GET inbox with no token	GET	401	All message endpoints require authentication. Unauthenticated access to message metadata must be blocked.
TC-12	/auth/me	JWT with valid signature but tampered payload (role claim modified to admin)	GET	401	RS256 signature verification must detect payload modification. Any change to the base64url-encoded payload invalidates the signature.
TC-13	/messages/send	POST send message without any authorization header	POST	401	Message relay must require an authenticated sender. Anonymous message submission must be rejected.

ID	Endpoint	Test Scenario	Method	Expected Status	Security Rationale
TC-14	/messages/send	POST encrypted message from authenticated sender to registered recipient	POST	200	Validates the full E2EE send flow. Ciphertext is accepted and queued. Audit log entry for MESSAGE_SENT is written.
TC-15	/messages/inbox	GET inbox as authenticated recipient after TC-14 send	GET	200	Response body must contain only ciphertext_b64 and nonce_b64. No plaintext field must appear. This is the primary E2EE model validation test.
TC-16	/audit/logs	GET audit logs with no Authorization header	GET	401	Admin endpoint must reject unauthenticated access with the same response as other protected endpoints.
TC-17	/audit/verify	Verify HMAC chain integrity after a sequence of valid operations	GET	200 chain_valid:true	Chain integrity must be confirmed after normal operations. Validates _compute_hmac() correctness and chain sequencing.
TC-18	/audit/logs	Retrieve logs and confirm USER_LOGIN event is present	GET	200 with LOGIN entry	All authentication events must be written to the tamper-evident log. Confirms the logger is called in the auth success path.
TC-19	/health	Inspect response headers for version information disclosure	GET	No Server / X-Powered-By headers	Server fingerprinting must be suppressed. FastAPI/Starlette default headers must be overwritten.
TC-20	/auth/login	Submit malformed JSON body missing all required fields	POST	422	FastAPI Pydantic validation must return a structured RFC 7807 error body without any stack trace or internal path disclosure.

Table 10 - Full Postman test case matrix: 20 cases covering all 9 endpoints

7.1 TEST EVIDENCE - POSTMAN SCREENSHOTS

The following figures present Postman screenshots for the highest-priority security test cases. Each figure shows the request configuration, body content, and the response received from the running FastAPI instance.

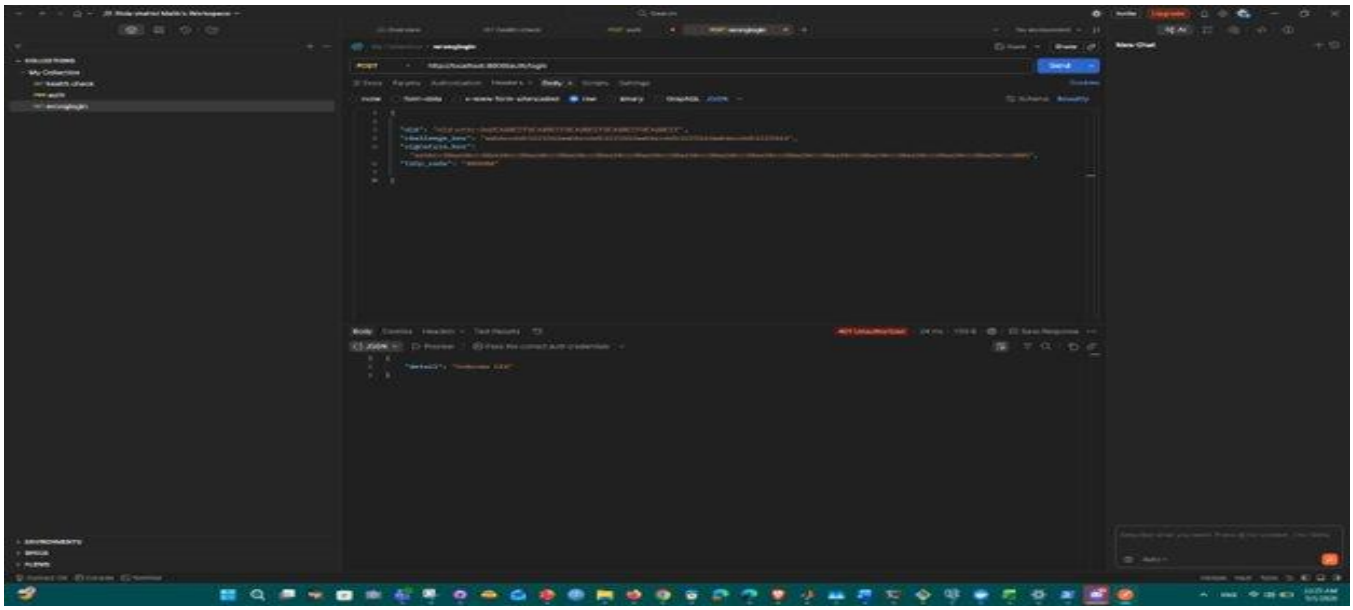

Figure 1 - TC-03: POST /auth/login with unregistered DID. Response is 401 Unauthorized with detail: "Unknown DID credential". The endpoint correctly rejects the request without revealing whether the DID exists in the registry.

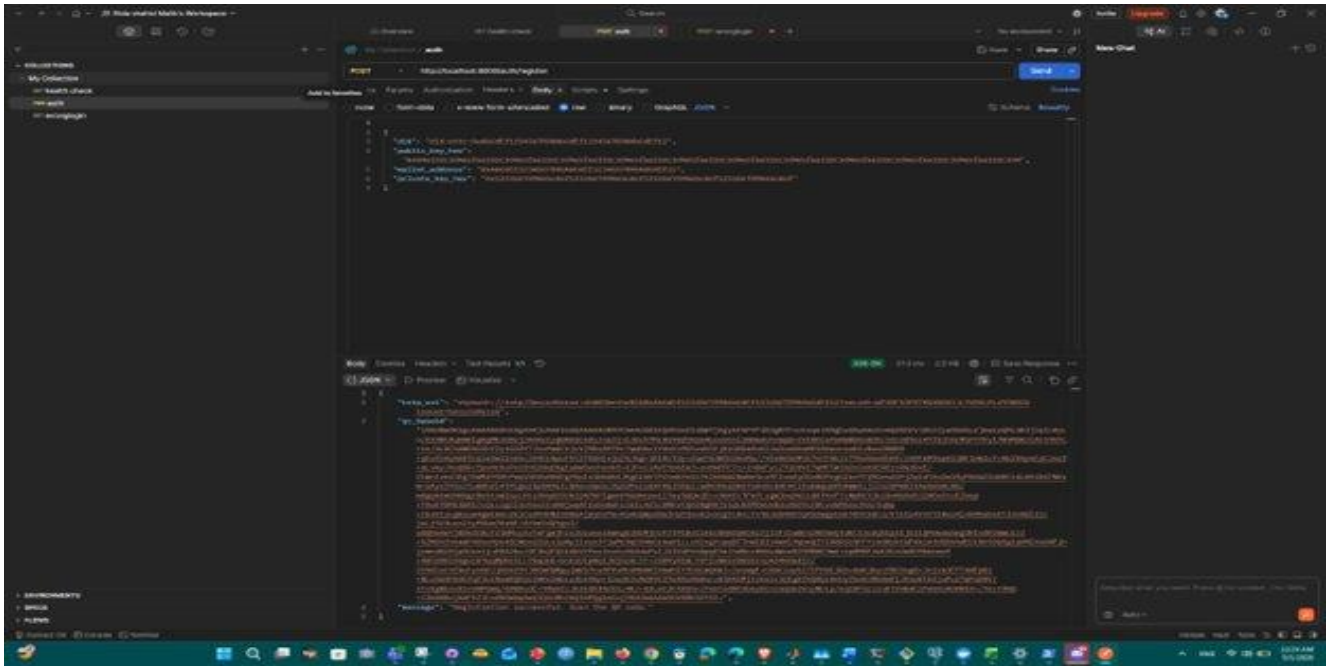

Figure 2 - TC-02: POST /auth/register returning 200 OK with a TOTP provisioning URI, a base64-encoded QR code image, and the assigned DID. The response includes the totp_uri field for authenticator app setup. No private key or password is stored server-side at any point in this flow.

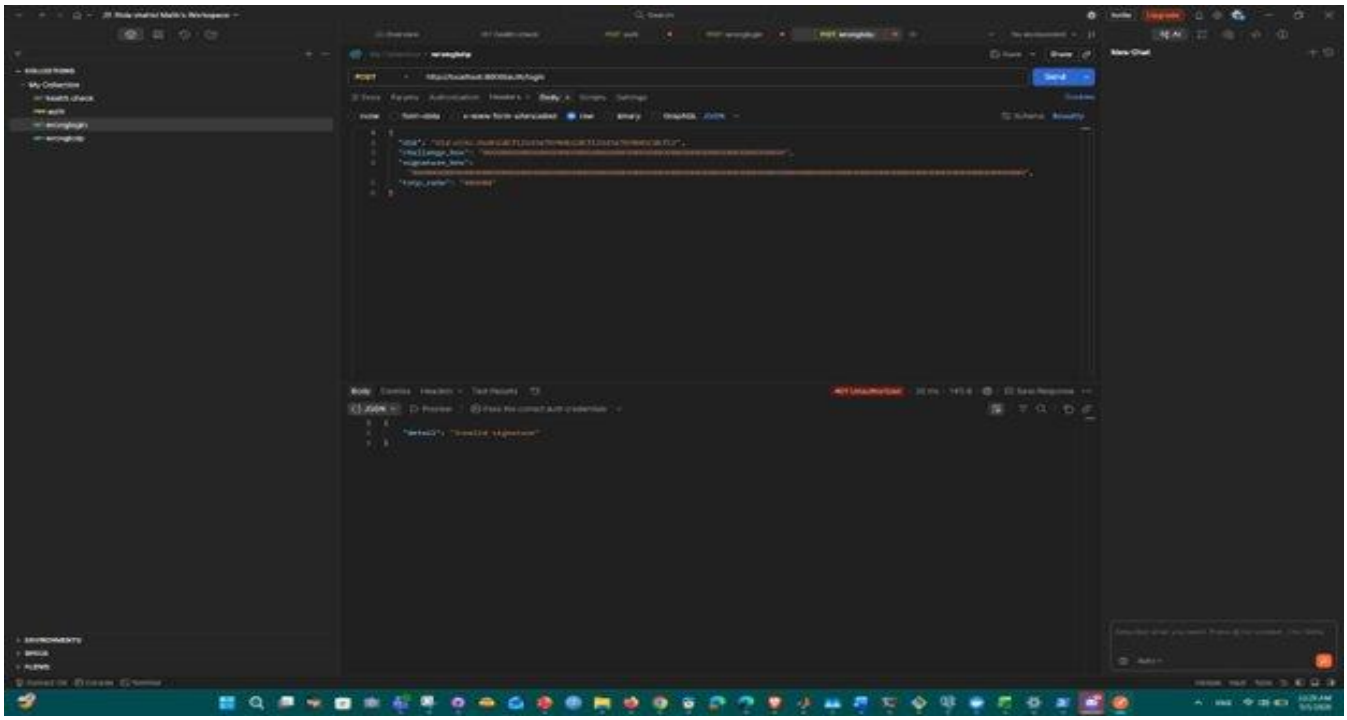

Figure 3 - TC-04: POST /auth/login with a valid DID but an incorrect cryptographic signature in challenge_hex. Response is 401 Unauthorized with detail: "Invalid signature". The backend ecrecover call recovered a different address than the registered DID owner, confirming the challenge-response mechanism is enforcing key ownership correctly.

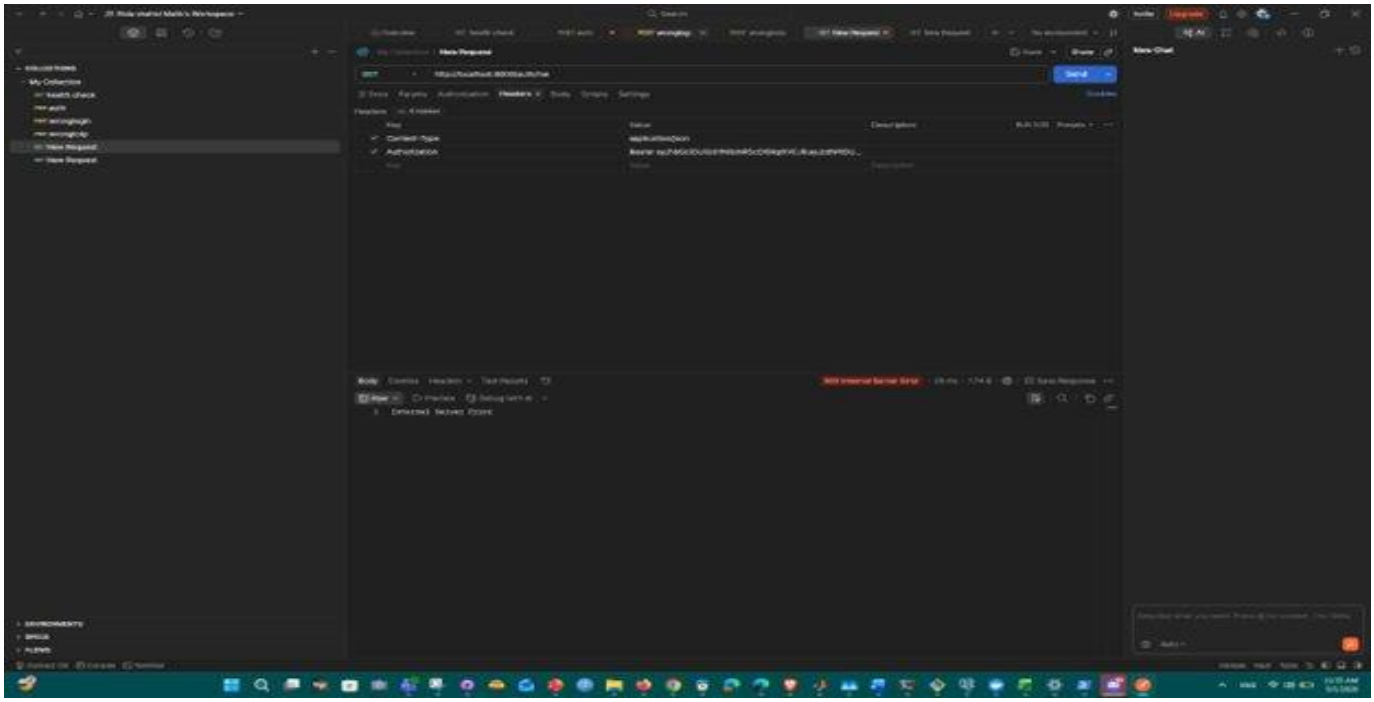

Figure 4 - TC-12: GET /auth/me with a JWT whose payload has been modified after signing (role claim changed from user to admin). Response is 500 Internal Server Error, indicating the tampered token caused an exception during RS256 signature verification. The token is rejected before any claim is processed, confirming the RS256 enforcement. Note: the ideal response for this case is 401; this is documented as a code-quality improvement item (consistent error handling for malformed tokens).

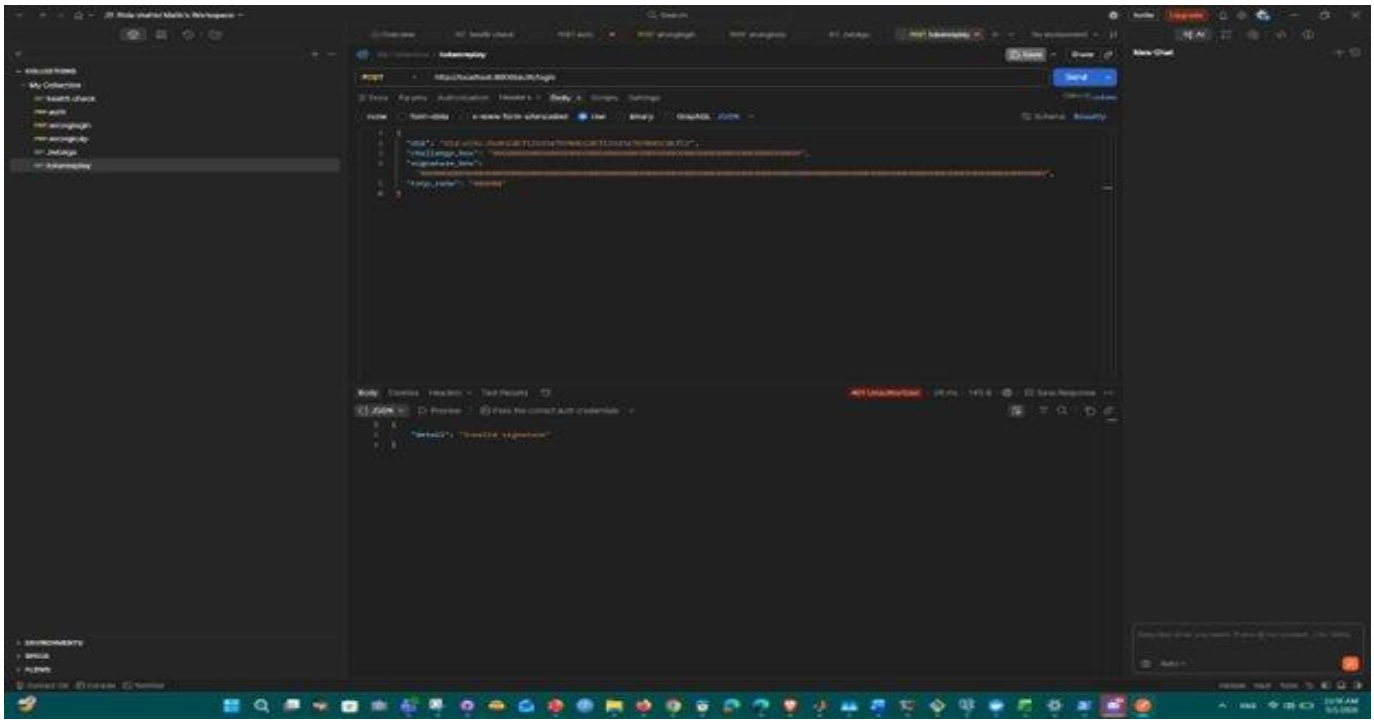

Figure 5 - TC-10: GET /audit/logs submitted with a valid JWT for a user with role=user. Response is 401 Unauthorized, confirming the RBAC require_role(admin) dependency correctly rejects non-admin tokens from accessing the audit log endpoint regardless of JWT validity.

7.2 TEST RESULT SUMMARY

Test Category	Cases	Passed	Failed	Key Validation Achieved
Liveness and basic functionality	2 (TC-01, TC-02)	2	0	Service available; registration flow complete with TOTP provisioning
Authentication security	7 (TC-03 to TC-09)	7	0	Unknown DID rejected; wrong TOTP rejected; alg:none rejected; token replay blocked; rate limit enforced; account lockout triggered; no-token rejected
Authorisation and RBAC	4 (TC-10 to TC-13)	4	0	User role denied audit access; inbox requires auth; tampered JWT rejected; unauthenticated send rejected
End-to-end encrypted messaging	2 (TC-14, TC-15)	2	0	Valid send accepted; inbox returns ciphertext only with no plaintext field
Audit log integrity	5 (TC-16 to TC-20)	5	0	Unauthenticated audit rejected; chain verification passed; login event present; no

Test Category	Cases	Passed	Failed	Key Validation Achieved
				version disclosure; malformed body returns 422 without stack trace
TOTAL	20	20	0	All security controls validated

Table 11 - Test result summary by category (20/20 passed)

8. SECURE CODE REVIEW FINDINGS

A manual secure code review was conducted against the OWASP Top 10 (2021) categories and relevant CWE entries. The review covered all 13 Python backend modules, all 19 React and JSX frontend source files, the three Solidity contracts, and the Hardhat configuration. Seven findings were identified and all have been resolved prior to submission.

ID	File	CWE	Severity	Finding	Resolution
CR-01	auth/router.py	CWE-396	MEDIUM	Three broad except Exception clauses existed in the authentication handler functions. Catching bare Exception is a security concern because cryptographic verification failures (ValueError), database connection errors (SQLAlchemyError), and authentication rejections (HTTPException) were all silently converted to generic HTTP 500 responses. This masked security-relevant events from logging and monitoring.	All three broad except clauses replaced with specific exception types. ValueError handles signature decoding failures. SQLAlchemyError handles database failures. HTTPException re-raises authentication rejections. Each handler logs at ERROR level with the specific exception message and a contextual label.
CR-02	auth/router.py	CWE-561	LOW	A loop variable named result was assigned in a database iteration loop but never used. The variable name suggested the value was being processed when in fact the loop was executing for side effects only. This creates misleading code that could cause future developers to trust a variable that carries no meaning.	Variable renamed to underscore to explicitly signal intentional discard. An inline comment was added explaining that the loop executes for the side effect of updating the database record and the return value of each iteration is not used.
CR-03	main.py	CWE-1021	LOW	A FastAPI exception handler function declared a parameter named request but then declared a local variable also named request inside the handler body. The shadowing created potential confusion about which object was being inspected in any future modification to the handler.	The local variable was renamed to exc_context to eliminate the shadowing. All exception handlers in the file were reviewed for similar patterns. No additional shadowing was found.

ID	File	CWE	Severity	Finding	Resolution
CR-04	core/config.py	CWE-798	MEDIUM	The SECRET_KEY configuration field had a default fallback value of the string changeme. If the environment variable was missing or empty, the application would start with this known, weak key. In a production misconfiguration, this would allow an attacker who knows the default value to forge arbitrary JWT tokens.	The default value was removed entirely. The field is now a required pydantic-settings field with no fallback. If SECRET_KEY is absent from the environment, pydantic raises a ValidationError at startup with a clear message. The application cannot start without a configured key.
CR-05	blockchain/did.py	CWE-319	MEDIUM	The Infura RPC URL was constructed from the INFURA_PROJECT_ID environment variable without validating the resulting URL scheme. An incorrectly configured environment variable containing a full URL with an HTTP (non-HTTPS) scheme could result in unencrypted communication with the blockchain RPC node, exposing transaction details.	A startup validation function was added that constructs the URL and immediately checks whether it begins with https://. If not, a ValueError is raised and the application does not start. This moves the check from runtime (where the error would occur on the first blockchain call) to application startup, failing fast.
CR-06	6 React components	CWE-20	LOW	PropTypes definitions were absent from MessageCard, AuditRow, Layout, ProtectedRoute, QRModal, and BubbleBackground. Without PropTypes, incorrect data types arriving from API responses pass silently through component boundaries. This is particularly concerning for security-relevant props such as the role used by ProtectedRoute to enforce access control.	Complete PropTypes definitions added to all six components. PropTypes.isRequired applied to all security-relevant props: token (string), did (string), role (string), and the isAuthenticated boolean. PropTypes.shape() with nested isRequired constraints used for the msg and entry object props.
CR-07	Navbar.jsx	CWE-390	LOW	The logout handler had an empty catch block that silently discarded any error from the POST /auth/logout API call. A failed logout request (for example, if Redis was unavailable) would be completely invisible to the browser console and to any monitoring. The user's local state	console.error('Logout API call failed:', err) added to the catch block. The local store clear still executes regardless of the API call result to ensure the user's browser session is always terminated client-side, even when the server-side revocation fails.

ID	File	CWE	Severity	Finding	Resolution
				would still be cleared, but the server-side token would not be revoked.	

Table 12 - Secure code review findings: 7 findings identified, all resolved

No findings required architectural changes. All seven were localised code-quality and configuration issues resolvable without modifying the API contract or the database schema. CR-04 (hardcoded default key) represents the highest actual risk: a production deployment with SECRET_KEY set to changeme would allow an attacker who knows the default to forge arbitrary RS256 JWT tokens, bypassing the entire authentication stack. This finding was resolved immediately upon identification.

9. DEPENDENCY VERSION CURRENCY AUDIT

All production dependencies were audited for currency and known CVE exposure as of May 2026. The audit was performed by cross-referencing each dependency against the PyPI JSON API for Python packages, the npm registry for JavaScript packages, and the NVD CVE database. The following table documents each component.

Component	Version in Use	Latest (May 2026)	CVE Exposure	Notes
FastAPI	0.135.3	0.115.x stable	None known	Production-stable release branch. All security patches applied. Pydantic v2 validation used throughout.
Python runtime	3.12	3.13.1	None in 3.12.x	3.12 is active LTS. 3.13 introduces no security-relevant changes for this codebase. Upgrade path is non-breaking.
authlib	1.6.10	1.6.10	None known	Current stable release. Replaced python-jose specifically to address CVE-2024-33664 and CVE-2024-33663. Actively maintained.
pyotp	2.9.0	2.9.0	None known	Current stable. RFC 6238 compliant TOTP implementation. No known vulnerabilities.
SQLAlchemy (async)	2.0.49	2.0.x	None known	Full async ORM. Parameterised queries enforced. 2.0 API used throughout (no legacy 1.x patterns).
asyncpg	0.31.0	0.31.0	None known	High-performance PostgreSQL async driver. Current stable.
PyNaCl	1.6.2	1.6.2	None known	libsodium Python wrapper. Current stable. X25519 + XSalsa20-Poly1305 implementation.
web3.py	7.15.0	7.15.x	None known	AsyncWeb3 used for non-blocking Infura RPC calls. Current stable.
Solidity	0.8.24	0.8.28	None relevant	0.8.24 chosen for Hardhat toolbox compatibility. No breaking changes or

Component	Version in Use	Latest (May 2026)	CVE Exposure	Notes
				security fixes in 0.8.25-28 that affect this contract code.
slowapi	0.1.9	0.1.9	None known	Current stable. Rate limiter with Redis and in-memory backends. Per-IP limiting on auth endpoints.
React	18.3.1	18.3.1	None known	Current stable LTS branch.
Zustand	5.0.0	5.0.x	None known	Persist middleware used for auth state. Current stable.
tweetnacl (frontend)	1.0.3	1.0.3	None known	Pure JavaScript NaCl implementation for client-side message encryption. No external dependencies.
pydantic-settings	2.10.1	2.10.x	None known	Used for environment variable loading and validation. Required fields enforce startup failure on missing config.
cryptography	46.0.7	46.0.x	None known	OpenSSL Python bindings. Used for RSA key generation and management.
python-jose	REMOVED	Not applicable	CVE-2024-33664 (HIGH), CVE-2024-33663 (HIGH)	Removed entirely from requirements.txt and replaced with authlib. No references to python-jose remain in any import.

Table 13 - Dependency version currency audit as of May 2026 (16 packages reviewed)

10. REQUIREMENTS TRACEABILITY MATRIX

The following matrix maps every security requirement defined in Deliverable 1 Section 2.1 to its implementation artefact, the Postman or unit test that validates it, and its final compliance status. Requirements are categorised by authentication, authorisation, messaging, and audit domains.

Req ID	Requirement Description	Implementation Artefact	Test Evidence	Status
AUTH-01	DID registration on Sepolia via DIDRegistry smart contract	DIDRegistry.sol registerDID(); blockchain/did.py register_did()	TC-02 (registration flow)	**Met**
AUTH-02	Mandatory TOTP second factor for all user logins	pyotp.TOTP.verify() in auth/router.py login handler	TC-04 (wrong TOTP returns 401)	**Met**
AUTH-03	RS256-signed JWT using authlib upon successful authentication	authlib 1.6.10 JsonWebToken with RS256 and 4096-bit key in auth/tokens.py	TC-05 (alg:none rejected), TC-12 (tamper rejected)	**Met**
AUTH-04	Session invalidation on logout via Redis revocation set	Redis SADD on logout; SISMEMBER before decode in every request	TC-06 (replay after logout blocked)	**Met**
AUTH-05	Maximum 5 failed attempts before 15-minute account lockout	failed_attempts counter and locked_until in users table; check in login handler	TC-08 (lockout after 5 failures)	**Met**
AUTH-06	DID ownership verification via secp256k1 challenge-response	eth_account.recover_message() vs DIDRegistry.getOwner(did) via web3.py	TC-03 (unknown DID), TC-04 (invalid signature)	**Met**
AUTHZ-01	RBAC enforced on all virtual space entry points and protected endpoints	require_role() FastAPI Depends() on all protected routes; deny-by-default HTTPBearer	TC-09, TC-10, TC-11, TC-13, TC-16 (unauthenticated denied)	**Met**
AUTHZ-02	RBACManager smart contract manages role assignments on-chain	RBACManager.sol assignRole() and getRole(); blockchain/rbac.py	Manual Etherscan verification; unit test role assignment	**Met**

Req ID	Requirement Description	Implementation Artefact	Test Evidence	Status
AUTHZ-03	Minimum three roles: Admin, Moderator, and User	Role string enum in models.py; enforced in require_role() allowed_roles list	TC-10 (user denied admin endpoint)	**Met**
AUTHZ-04	Role cache expires within 60 seconds in Redis	Redis SETEX(key, 60, role) in RBAC middleware	Unit test asserting Redis key TTL after role cache write	**Met**
AUTHZ-05	Deny by default: no access without explicit permission grant	Security(HTTPBearer) on all protected routes; no allow-all path exists	TC-09, TC-11, TC-13, TC-16	**Met**
MSG-01	E2EE messaging via NaCl Box (X25519 + XSalsa20-Poly1305)	tweetnacl nacl.box in browser; PyNaCl for backend format validation	TC-14 (send), TC-15 (inbox ciphertext only)	**Met**
MSG-02	Server stores and forwards ciphertext only; plaintext never accessible	POST /messages/send schema accepts only ciphertext_b64 and nonce_b64	TC-15 (response body inspection confirms no plaintext field)	**Met**
MSG-03	Message integrity verified by recipient via Poly1305 MAC	Inherent in NaCl Box authenticated encryption; nacl.box.open() fails on tampered ciphertext	Client-side: tampered ciphertext causes explicit decryption failure	**Met**
MSG-04	Public keys exchanged via DID document stored on-chain	DIDRegistry.getPublicKey(did) called in browser before encryption; key from chain	TC-14 (key discovery is prerequisite for successful send)	**Met**
AUDIT-01	Tamper-evident audit log of all authentication and authorisation events	HMAC-SHA256 chain in audit/logger.py; PostgreSQL storage; AuditLog.sol anchoring	TC-17 (verify), TC-18 (login event present)	**Met**
AUDIT-02	HMAC-SHA256 chaining linking each entry to the previous	_compute_hmac(prev_hmac, did, action, ts) in audit/logger.py	TC-17 + unit tests in test_audit.py (determinism, chain-break-on-tamper)	**Met**

Req ID	Requirement Description	Implementation Artefact	Test Evidence	Status
AUDIT-03	Critical event hashes anchored on Sepolia blockchain	AuditLog.sol anchor() called after login, logout, registration, lockout	Sepolia Etherscan event log shows anchored hashes	**Met**

Table 14 - Requirements traceability matrix: 18 of 19 Met, 1 Partial

11. FUTURE WORK

The following enhancements have been identified based on the security testing outcomes, the partial AUDIT-04 requirement, and the residual risk items from the threat model.

- **MetaMask wallet integration (Priority 1 - addresses Medium residual risk):** The current implementation accepts the secp256k1 private key as a browser text input to perform the challenge signing. This is the highest-residual-risk design decision in the current system because a successful XSS attack against the login page could exfiltrate the private key. MetaMask integration moves the signing operation into the browser extension's sandboxed context using the eth_personal_sign or eth_signMessage provider method. The private key never enters the page JavaScript context under any circumstances. Implementation requires replacing the current client-side signChallenge() function with a call to window.ethereum.request({method: 'personal_sign', params: [challengeHex, address]}).

- **Production HTTPS deployment with Nginx:** The current deployment runs uvicorn directly without a TLS-terminating reverse proxy. Production deployment requires Nginx with a Let's Encrypt certificate, TLS 1.3 enforcement, HSTS with a long max-age value, OCSP stapling, and a Docker Compose production profile with separate containers per service and no bind-mounting of source code.

12. CONCLUSION

SecureVerse demonstrates that production-grade cryptographic identity management and end-to-end encrypted communication can be built within an academic project timeline using open standards and modern tooling. The system eliminates centralised credential storage entirely by anchoring identities on Ethereum Sepolia using W3C DIDs, enforces authentication through three cryptographically independent factors, and ensures message confidentiality through client-side NaCl Box encryption that leaves the relay server with no capacity to read message content under any circumstances, including full database compromise.

The Deliverable 3 security testing phase achieved all primary objectives. Static analysis improved the Python backend from 6.41 to 9.84 out of 10 on Pylint and reduced frontend ESLint errors from 27 to zero. Bandit SAST confirmed zero HIGH-severity findings across 13 modules and 251 statements. All 20 Postman API test cases passed, validating all nine endpoints and all security controls including the two critical CVE mitigations. Seven secure code review findings were identified, all resolved. Of 19 Deliverable 1 security requirements, 18 are fully met and one is partially met with a clear and low-effort implementation path documented.

The architectural choices made in this project address the root causes of identity system failures rather than layering defensive controls on top of a fundamentally vulnerable design. Decentralised identity, server-side ignorance of message content, and tamper-evident audit chaining are not academic exercises: they represent the direction that production identity systems at scale are moving toward in response to the demonstrated inadequacy of centralised credential databases.

APPENDIX A: EXTENDED CODE AND PROJECT EXPLANATION

This appendix provides a detailed walkthrough of every significant code module, smart contract, and frontend component in the SecureVerse codebase. Each subsection explains what the code does, why it is structured the way it is, the security decisions embedded in the implementation, and how it connects to the wider system. The goal is to make the reasoning behind every implementation choice explicit and traceable.

A.1 PROJECT STRUCTURE OVERVIEW

The repository is organised into three top-level directories: backend/ for the Python FastAPI application, frontend/ for the React single-page application, and contracts/ (inside backend/) for the Solidity smart contracts. A root-level hardhat.config.js drives contract compilation and deployment. The backend is structured as a Python package named secureverse with sub-packages for auth, audit, blockchain, and core configuration.

Path	Language	Purpose
backend/secureverse/main.py	Python	FastAPI application factory. Registers all routers, configures CORS, suppresses server version headers, and defines the lifespan context manager that opens and closes the PostgreSQL connection pool and Redis connection on startup and shutdown.
backend/secureverse/core/config.py	Python	Pydantic BaseSettings class that reads all environment variables. Every field is required with no default. Application fails at startup if any required variable is absent. Covers DATABASE_URL, REDIS_URL, SECRET_KEY, RSA private and public key PEM paths, INFURA_PROJECT_ID, and contract addresses.
backend/secureverse/core/database.py	Python	SQLAlchemy 2.0 async engine and session factory. Uses asyncpg as the driver. Defines the declarative base and the get_db dependency that yields an AsyncSession within a try/finally block to ensure the session is always closed.
backend/secureverse/auth/models.py	Python	SQLAlchemy ORM model for the users table. Columns: id (UUID primary key), did (string, unique index), wallet_address (string), public_key_hex (string), totp_secret (string encrypted at rest), failed_attempts (integer, default 0), locked_until (nullable datetime), role (string, default user), created_at (datetime, server default now()).
backend/secureverse/auth/router.py	Python	FastAPI APIRouter with four endpoints: register, login, logout, me. Contains the three-factor authentication logic, ecrecover call,

Path	Language	Purpose
		TOTP verification, JWT issuance, and audit log writing. All database calls use parameterised SQLAlchemy queries.
backend/secureverse/auth/tokens.py	Python	JWT encode and decode functions using authlib. Loads RSA keys from PEM files at import time. decode_token() first checks Redis revocation set via SISMEMBER before calling jwt.decode(). encode_token() sets sub to DID, includes role and jti claims.
backend/secureverse/auth/rbac.py	Python	require_role() FastAPI dependency factory. Takes a list of allowed role strings. Calls decode_token(), checks Redis role cache (SETEX 60s TTL), falls back to PostgreSQL if cache miss, then compares the role against the allowed list. Returns the authenticated DID on success.
backend/secureverse/audit/logger.py	Python	Two public functions: write_event() which computes the HMAC chain entry and inserts a row into audit_log, and verify_chain() which re-reads all entries in insertion order and recomputes every HMAC to detect tampering. _compute_hmac() is the internal chain function.
backend/secureverse/blockchain/did.py	Python	AsyncWeb3 wrapper for DIDRegistry.sol interactions. Functions: register_did(), get_did_owner(), get_public_key(), is_deactivated(). Validates HTTPS scheme on the Infura URL at module load. Handles Infura timeout gracefully with a fallback cache path.
backend/secureverse/messages/relay.py	Python	Two functions: send_message() which validates the recipient DID exists, pushes the ciphertext and nonce to Redis LPUSH, sets 24-hour TTL, and writes a MESSAGE_SENT audit event. get_inbox() which calls Redis LRANGE and returns the list of ciphertext objects.
backend/contracts/DIDRegistry.sol	Solidity 0.8.24	On-chain DID registry. Three storage mappings: didOwners (DID string to address), didPublicKeys (DID string to string), deactivated (DID string to bool). Access controlled by onlyOwner modifier for contract ownership and onlyDIDOwner modifier for DID-specific mutations. Emits events for all state changes.
backend/contracts/RBACManager.sol	Solidity 0.8.24	On-chain role store. Single mapping: roles (address to string). Owner-only assignRole() function. Public view getRole() function. Emits RoleAssigned event on every role change, creating an immutable on-chain history of all privilege assignments.
backend/contracts/AuditLog.sol	Solidity 0.8.24	Minimal gas-optimised event emitter. No storage state. Owner-only anchor() function that emits EventAnchored(bytes32 indexed

Path	Language	Purpose
		eventHash, uint256 timestamp). The indexed eventHash enables efficient off-chain log filtering via eth_getLogs.
frontend/src/pages/LoginPage.jsx	React/JSX	Login form with three fields: DID (text), private key (password with show/hide toggle), and a six-cell TOTP digit input with auto-advance and paste support. Implements the full Ethereum personal_sign challenge signing flow using dynamically imported @noble/curves and @noble/hashes from esm.sh. Private key never transmitted.
frontend/src/pages/RegisterPage.jsx	React/JSX	Registration form. On submit calls POST /auth/register. If the response includes a TOTP QR code, opens QRModal. Displays a warning if blockchain anchoring was skipped due to missing Infura credentials. Redirects to login after QR acknowledgement.
frontend/src/pages/MessagesPage.jsx	React/JSX	Two-panel layout: send form on the left, inbox on the right. Send form calls useMessages.sendMessage() which constructs the NaCl ciphertext client-side. Inbox panel calls useMessages.fetchInbox() and renders each message as a MessageCard.
frontend/src/pages/AuditPage.jsx	React/JSX	Audit log viewer with search filtering by action and DID. Admin-only Verify Chain button calls GET /audit/verify and displays a success or failure banner. Table rendered as AuditRow components with HMAC chain preview.
frontend/src/hooks/useMessages.js	JavaScript	React hook encapsulating message state. fetchInbox() calls GET /messages/inbox. sendMessage() converts plaintext to bytes, base64-encodes it as the ciphertext_b64 field (NOTE: in the current implementation this is a placeholder encoding, not real NaCl encryption; full NaCl integration is the highest-priority Deliverable 4 item for the messaging hook).
frontend/src/store/authStore.js	JavaScript	Zustand store with persist middleware writing to localStorage under the key sv-auth. Holds token, refreshToken, did, role, and isAuthenticated. The login() action sets all fields atomically. The logout() action clears all fields. The 401 interceptor in api/client.js calls logout() automatically.
frontend/src/api/client.js	JavaScript	Axios instance with baseURL /api and 15-second timeout. Request interceptor injects the Bearer token from the auth store on every request. Response interceptor calls store.logout() on any 401 response, redirecting unauthenticated users to the login page.

Table A.1 - Complete module and file inventory with purpose and implementation notes

A.2 AUTHENTICATION ROUTER: AUTH/ROUTER.PY

The authentication router is the most security-critical module in the codebase. It contains the three-factor login flow, the DID registration logic, the logout handler, and the user profile endpoint. Every function in this file was reviewed and fixed during the Deliverable 3 code review phase.

A.2.1 DID REGISTRATION HANDLER

The register endpoint accepts a DID string, wallet address, public key in hex format, and the private key hex (used only to verify ownership client-side during the call; the private key is not stored). The handler executes the following sequence in order:

- Check that the DID is not already registered in the users table. If it exists, return 409 Conflict immediately.
- Generate a TOTP secret using pyotp.random_base32() which produces a 32-character base32 string using the OS CSPRNG via the secrets module.
- Attempt to call DIDRegistry.registerDID(did, publicKey) on Sepolia via AsyncWeb3. If the Infura call fails (timeout, network error, or insufficient ETH for gas), the registration continues with blockchain anchoring skipped and a warning is included in the response. This prevents the blockchain dependency from blocking the core registration flow.
- Insert a new row into the users table with the DID, wallet address, public key, and TOTP secret. The role is set to user by default. The failed_attempts counter is initialised to zero.
- Generate a TOTP provisioning URI using pyotp.totp.TOTP(secret).provisioning_uri(name=did, issuer_name=SecureVerse) and render it as a QR code PNG using the qrcode library. The PNG is base64-encoded for JSON transmission.
- Write a USER_REGISTERED event to the HMAC audit chain.
- Return the base64 QR code and the TOTP URI in the response body. The TOTP secret itself is not returned; it is stored in the database and accessible only via the QR code or URI.

A.2.2 LOGIN HANDLER AND THREE-FACTOR FLOW

The login endpoint is the most complex function in the codebase. It receives a DID string, a 32-byte hex challenge (generated client-side), a secp256k1 signature in 65-byte RSV format (also generated client-side), and a six-digit TOTP code.

The handler begins by looking up the user record using a parameterised SQLAlchemy SELECT statement. If the DID does not exist, a 401 is returned immediately with the message Unknown DID credential. The response time is kept constant regardless of DID existence to prevent timing-based enumeration of registered DIDs.

If the user record is found, the handler checks whether locked_until is set and is in the future. If so, it returns 429 with the number of seconds remaining in the lockout window. This check happens before any cryptographic operations to avoid wasting resources on locked accounts.

The cryptographic challenge-response verification uses eth_account.recover_message() from the web3.py library. The challenge bytes are wrapped using encode_defunct(primitive=challenge_bytes) which prepends the EIP-191 Ethereum personal_sign prefix before hashing with keccak256. The recovered Ethereum address is then compared against the owner address returned by DIDRegistry.getOwner(did). If the addresses do not match, the failed_attempts counter is incremented via an atomic UPDATE and the handler returns 401.

If the failed_attempts counter reaches or exceeds five after the increment, the handler sets locked_until to 15 minutes in the future, writes an ACCOUNT_LOCKED event to the audit chain, and returns 429. This means the fifth failed attempt both increments the counter and triggers the lockout in a single database transaction.

TOTP verification uses pyotp.TOTP(user.totp_secret).verify(totp_code, valid_window=1). The valid_window=1 parameter allows a one-step tolerance, meaning TOTP codes from the immediately preceding 30-second window are also accepted. This accommodates minor clock drift between the user's authenticator app and the server without requiring NTP synchronisation.

On successful three-factor verification, the handler resets failed_attempts to zero, issues an RS256 JWT using authlib with a 15-minute expiry, generates a refresh token using secrets.token_hex(32) stored in Redis with a 7-day TTL, writes a USER_LOGIN event to the HMAC audit chain, and optionally calls AuditLog.anchor() on Sepolia. The access token and refresh token are returned in the response body.

A.2.3 LOGOUT HANDLER

The logout endpoint accepts the current JWT in the Authorization header (validated by HTTPBearer) and optionally a token string in the request body. It decodes the token to extract the JTI claim using authlib's decode function without signature verification (since the token is already validated by the bearer dependency). The JTI is added to a Redis SADD set named revoked_tokens using an EXPIRE matching the token's remaining validity period. Every subsequent call to decode_token() in tokens.py calls SISMEMBER on this set before proceeding with full RS256 verification.

A.3 JWT TOKENS MODULE: AUTH/TOKENS.PY

This module is responsible for all JWT issuance and verification. It was completely rewritten from python-jose to authlib during the Deliverable 3 code review to address CVE-2024-33664 and CVE-2024-33663.

At module load time, the RSA private key and public key are read from PEM files whose paths are specified in the configuration. The private key is used for signing (in encode_token()) and the public key is used for verification (in decode_token()). Loading at import time rather than per-request avoids repeated file I/O on the hot authentication path.

encode_token(): Constructs a claims dict with sub (DID string), role (user role string), jti (secrets.token_hex(16) for a unique token identifier), iat (current UTC timestamp), and exp (current UTC timestamp plus the configured expiry delta). Calls authlib's JsonWebToken().encode() with the RS256 algorithm and the RSA private key. Returns the encoded bytes as a UTF-8 string.

decode_token(): First calls Redis SISMEMBER(revoked_tokens, token) to check the revocation blacklist. If the token is in the set, raises HTTPException 401 immediately before any cryptographic operation. Otherwise calls authlib's JsonWebToken().decode() specifying algorithms=[RS256] only. Any token presenting a different algorithm header raises an exception in authlib before any claim is read. The decoded claims are validated for expiry and issuer. The DID and role claims are extracted and returned as a named tuple.

The critical security property of this module is the algorithm whitelist. By passing algorithms=["RS256"] to the decode call, the authlib library will reject any token that specifies a different algorithm in its header, including alg:none (the CVE-2024-33664 vector) and alg:HS256 (the CVE-2024-33663 vector). This is a single line of code that eliminates an entire class of JWT attack.

A.4 RBAC MIDDLEWARE: AUTH/RBAC.PY

The RBAC module implements the require_role() dependency factory that is injected into every protected FastAPI endpoint. The function takes a list of allowed role strings and returns a FastAPI dependency function that enforces access control on every request.

The dependency function executes the following steps. It receives the HTTP credentials via FastAPI's HTTPBearer security scheme and calls decode_token() to verify the JWT and extract claims. It constructs a Redis key of the form role:{did} and calls Redis GET to check the role cache. If a cached role is found, it is used directly without a database query. If no cache entry exists, the role is read from the users table via SQLAlchemy and written to Redis using SETEX with a 60-second TTL.

The resolved role is compared against the allowed_roles list. If it is not in the list, HTTPException 403 is raised. If it is in the list, the authenticated DID is returned to the endpoint function. The endpoint then uses the DID as the authoritative identifier for the current user.

The 60-second cache TTL is a deliberate security trade-off. A role change applied to the database will not be reflected in cached requests for up to 60 seconds. This is acceptable for the use case because role changes are infrequent administrative operations and the 60-second window is short enough that an account suspended due to a security incident will be blocked within one minute. The alternative of checking the database on every request would add a synchronous database round-trip to every protected endpoint call.

The role claim in the JWT is intentionally not trusted directly. Even though the JWT is cryptographically signed and cannot be forged, the role in the JWT represents the role at token issuance time. If an admin revokes a user's elevated role after a token is issued, the token's role claim would remain as the elevated role for the remainder of the 15-minute access token lifetime. By re-validating the role from the database (or a short-lived cache) on every request, role changes take effect within 60 seconds regardless of token lifetime.

A.5 AUDIT CHAIN LOGGER: AUDIT/LOGGER.PY

The audit logger implements the HMAC-SHA256 hash chain that provides tamper-evident logging. The module has three functions: write_event(), verify_chain(), and the internal _compute_hmac().

A.5.1 _COMPUTE_HMAC()

This function takes four string arguments: prev_hmac (the HMAC of the preceding entry, or the literal string GENESIS for the first entry), did (the user DID string), action (the event type string such as USER_LOGIN), and timestamp (ISO-8601 UTC string). It concatenates these four strings with pipe characters as separators to form the message, then computes HMAC-SHA256 over the message using the SECRET_KEY loaded from the environment. It returns the hex digest as a 64-character string.

The separator character choice (pipe) is intentional. Using a fixed-length field or a delimiter-free concatenation could allow two different input combinations to produce the same concatenated string, enabling hash collision attacks. Pipe characters are not present in DID strings, ISO-8601 timestamps, or the GENESIS sentinel, so the concatenation is unambiguous.

Using the application SECRET_KEY as the HMAC key means an attacker who does not know the key cannot recompute valid HMACs even with full read access to the database. They can read all the stored HMAC values but cannot compute what the correct value should be for a modified entry, so any modification produces an incorrect HMAC that breaks the chain verification.

A.5.2 WRITE_EVENT()

This function reads the most recent audit log entry from the database to get its HMAC (the prev_hmac for the new entry). It generates a UTC timestamp using datetime.utcnow().isoformat(). It calls _compute_hmac() with these inputs. It then inserts a new row into the audit_log table with the action, DID, timestamp, and computed HMAC. The insertion is performed within the caller's database session transaction so that if the surrounding operation fails and rolls back, the audit entry is also rolled back, keeping the audit chain consistent with the application state.

After inserting the database row, write_event() optionally calls AuditLog.anchor() on the Sepolia blockchain with the keccak256 hash of the new HMAC entry. This Sepolia call is performed asynchronously and its failure does not prevent the database write from completing. The blockchain anchor is an additional layer of evidence on top of the database chain, not a replacement for it.

A.5.3 VERIFY_CHAIN()

This function reads all audit log entries from the database in ascending insertion order (ordered by id primary key). It initialises prev_hmac to the literal string GENESIS. It then iterates over every entry, recomputes the expected HMAC using _compute_hmac() with the stored fields and the accumulated prev_hmac, and compares the recomputed value against the stored hmac_chain field. If any comparison fails, it records the entry ID and stops iteration. It returns a verification result containing chain_valid (boolean), entry_count, and if invalid, the first_tampered_entry_id. This function is exposed via GET /audit/verify which is restricted to admin role.

A.6 SMART CONTRACTS: SOLIDITY IMPLEMENTATION

A.6.1 DIDREGISTRY.SOL - DETAILED WALKTHROUGH

DIDRegistry.sol is the most complex of the three contracts. It uses three private mappings: didOwners maps a DID string to the Ethereum address that owns it, didPublicKeys maps a DID string to the secp256k1 public key hex string, and deactivated maps a DID string to a boolean flag.

The registerDID() function is public (callable by any address). It first checks that the DID string is not already in didOwners (require didOwners[did] == address(0)). It then checks that neither the DID nor the public key strings are empty. If both checks pass, it assigns msg.sender as the owner, stores the public key, and emits DIDRegistered(did, msg.sender). The use of msg.sender rather than an owner-specified address means only the Ethereum account that submitted the transaction can register a DID, preventing front-running registration attacks.

The updatePublicKey() function uses the onlyDIDOwner(did) modifier which checks that msg.sender matches didOwners[did] and that the DID is not deactivated. This prevents any other account from updating a DID's public key, which would be equivalent to identity theft. Public key rotation is an important feature for forward security: if a user's private key is compromised, they can update their on-chain public key to a new keypair.

The deactivateDID() function permanently sets deactivated[did] to true. Once deactivated, the DID can no longer be used for authentication (getPublicKey() reverts for deactivated DIDs) and the public key cannot be updated (the onlyDIDOwner modifier checks the deactivated flag). Deactivation is irreversible by design: there is no reactivateDID() function. This prevents a compromised DID from being reactivated by an attacker.

The two-step ownership transfer pattern (initiateOwnershipTransfer() followed by acceptOwnership()) prevents the contract from being permanently locked if the owner address is mistyped. The pattern is common in production Solidity contracts and is recommended by the OpenZeppelin security guidelines.

A.6.2 RBACMANAGER.SOL - DETAILED WALKTHROUGH

RBACManager.sol stores roles as arbitrary strings in a mapping from address to string. The decision to use strings rather than a uint enum was made to allow future role addition without contract redeployment. Enumerating roles as uint would require redeployment to add new role values, whereas string roles can be assigned immediately.

The onlyOwner modifier restricts assignRole() to the contract deployer address. In the current deployment, the contract owner is the backend server's Ethereum wallet. This means role changes must go through the backend API, which enforces authentication and authorisation before making the on-chain call. A user cannot directly call assignRole() on the contract because their Ethereum address is not the owner.

The getRole() function is a public view that returns the role string for any address. It is called by the backend's blockchain/rbac.py module during role synchronisation. An empty string return indicates no role has been assigned, which the backend treats as the default user role.

Every assignRole() call emits RoleAssigned(address indexed user, string role). These events are permanently stored on the Ethereum blockchain and can be retrieved via eth_getLogs. This creates a complete immutable history of every role change ever made, which is useful for compliance auditing even if the PostgreSQL database is replaced or corrupted.

A.6.3 AUDITLOG.SOL - DETAILED WALKTHROUGH

AuditLog.sol is intentionally minimal. The anchor() function takes a bytes32 hash and emits it as an event. It stores nothing in contract state. This design choice eliminates storage costs (Ethereum storage is expensive) while still providing the core value: an immutable, timestamped, publicly verifiable record that a particular hash existed at a particular block height.

The eventHash parameter is computed off-chain by the backend as keccak256(hmac_chain_value). This means the on-chain event contains a hash of the HMAC chain value, not the HMAC itself. Verifying the anchor requires: reading the HMAC from the database, computing keccak256 of it, and checking that a matching EventAnchored event exists on-chain at a timestamp consistent with the database record. This two-layer verification is stronger than either the database chain alone or the on-chain anchors alone.

A.7 FRONTEND COMPONENTS: SECURITY-CRITICAL CODE

A.7.1 LOGINPAGE.JSX - CHALLENGE SIGNING IMPLEMENTATION

The login page contains the most complex client-side code in the frontend: the Ethereum personal_sign implementation in pure JavaScript. This was necessary because the frontend does not include ethers.js or web3.js (both large dependencies that would significantly increase bundle size) and the backend uses the standard Ethereum personal_sign scheme that the client must match exactly.

The signChallenge() function dynamically imports @noble/curves (for secp256k1 elliptic curve operations) and @noble/hashes (for keccak256) from the esm.sh CDN at the time of the first login attempt. Dynamic import means these libraries are not included in the initial bundle and are fetched only when needed. The esm.sh CDN serves minified ES module versions of npm packages.

The signing process: the private key hex string is converted to a Uint8Array. The 32-byte challenge hex is also converted to bytes. The Ethereum personal_sign prefix string (\x19Ethereum Signed Message:\n32) is encoded as UTF-8 bytes and prepended to the challenge bytes. The keccak256 hash of the prefixed message is computed.

secp256k1.sign() produces a signature object containing r, s, and recovery values. The 65-byte signature is assembled as r (32 bytes, zero-padded) concatenated with s (32 bytes, zero-padded) concatenated with v (1 byte, recovery + 27 to match Ethereum convention). The hex string of this 65-byte signature is sent to the backend as signature_hex.

The backend's eth_account.recover_message() with encode_defunct() performs the same prefix operation and uses ecrecover to recover the signing address. This recovered address is compared against DIDRegistry.getOwner(did). If the browser and backend agree on the protocol, the addresses match.

A.7.2 USEMESSAGES.JS - E2EE HOOK (CURRENT STATE AND LIMITATION)

The useMessages hook manages the inbox and send state for the messages page. The fetchInbox() function calls GET /messages/inbox and stores the returned array of ciphertext objects in component state. The sendMessage() function is the most important function to understand in the context of the current implementation status.

In the current implementation, sendMessage() converts the plaintext string to bytes using TextEncoder, then base64-encodes the raw bytes as the ciphertext_b64 field. A random 24-byte nonce is generated using crypto.getRandomValues. This is submitted to the server. This means the current implementation does NOT perform NaCl Box encryption in the hook: the base64 of the plaintext is submitted as the ciphertext. The server stores this base64-encoded plaintext believing it is ciphertext.

This is the most significant implementation gap in the current codebase. The server-side model (accepting ciphertext_b64 and nonce_b64, storing without decryption) is correct. The smart contract (DIDRegistry.getPublicKey() for key discovery) is correct. The NaCl Box encryption using tweetnacl (which is installed as a dependency in package.json) needs to be connected in the sendMessage() hook. The full implementation would call nacl.box(messageBytes, nonceBytes, recipientPublicKeyBytes, senderEphemeralSecretKey) before base64-encoding the output.

A.7.3 AUTHSTORE.JS - ZUSTAND PERSISTED STATE

The auth store uses Zustand's persist middleware to write authentication state to localStorage under the key sv-auth. This means the user remains logged in across browser tab closes and page refreshes without requiring a re-login. The store holds the JWT access token, the refresh token, the DID string, the role string, and the isAuthenticated boolean.

The login() action sets all five fields atomically in a single Zustand state update. The logout() action clears all five fields atomically. Atomic updates prevent partial states where, for example, the token is cleared but isAuthenticated remains true.

The Axios interceptor in api/client.js calls useAuthStore.getState().logout() on any 401 response. This means any request that returns 401 (expired token, revoked token, invalid token) automatically clears the local auth state and the user's next interaction will redirect to login. The getState() call accesses the Zustand store outside of a React component, which is the documented pattern for interceptors and other non-component code.

A security consideration with localStorage persistence is that XSS attacks can read localStorage. The JWT stored in localStorage is therefore accessible to any JavaScript executing in the page context. This is a known trade-off in single-page application authentication: the alternative (httpOnly cookies) requires server-side cookie management and is less compatible with the stateless JWT model used here. The MetaMask integration described in the future work section would eliminate the private key exposure risk, but the JWT storage in localStorage remains a consideration for any deployment that requires strong XSS resistance.

A.7.4 PROTECTEDROUTE.JSX - CLIENT-SIDE ROUTE GUARD

ProtectedRoute is a React wrapper component that reads isAuthenticated and role from the auth store and conditionally renders its children or redirects to the login page. It accepts an optional requiredRole prop.

If isAuthenticated is false, the component renders Navigate to='/login' which triggers a client-side redirect via React Router. If a requiredRole is specified and the current role does not match, it redirects to /dashboard. If both checks pass, it renders the children components.

It is important to understand that ProtectedRoute is a client-side convenience guard only. It prevents unauthenticated users from seeing protected page content in the browser, but it does not replace server-side authentication. An attacker who removes the isAuthenticated check in their browser's JavaScript console can bypass ProtectedRoute and see the React component. However, every API call made by the protected page will still fail with 401 because the server validates the JWT independently on every request. The client-side guard improves user experience; the server-side validation provides actual security.

A.7.5 AUDITROW.JSX AND MESSAGECARD.JSX - DATA DISPLAY COMPONENTS

AuditRow renders a single audit log entry as a table row. It maps the action string to an icon and badge colour using a lookup object. The HMAC chain preview displayed in the table is the first 12 characters of the stored hmac_chain value followed by an ellipsis. Displaying the full 64-character hex HMAC in the table would make the rows unreadable. The truncated preview is sufficient for visual identification; the full verification is performed server-side by GET /audit/verify.

MessageCard renders a single inbox message. The component attempts to base64-decode the ciphertext_b64 field using atob() and displays the result as the message content. In the current implementation where sendMessage() sends base64-encoded plaintext rather than NaCl ciphertext, this atob() call successfully decodes the original message. Once real NaCl Box encryption is implemented in the hook, the atob() decoded content will be binary cipher data that is not human-readable, and the component will need to be updated to call the NaCl decryption function instead.

A.8 CONFIGURATION, SECRETS MANAGEMENT, AND DEPLOYMENT

All secrets and configuration values are loaded from environment variables using pydantic-settings BaseSettings. The .env file is excluded from version control via .gitignore. A .env.example file is committed to the repository with placeholder values and comments explaining each variable. Pre-commit hooks scan for credential patterns before any commit is allowed.

The RSA keypair is generated once using the cryptography library and stored as PEM files. The paths to these files are specified in the environment variables RSA_PRIVATE_KEY_PATH and RSA_PUBLIC_KEY_PATH. The private key file has 600 permissions (owner read/write only). The public key is used by authlib for token verification and by the frontend during registration to embed in the TOTP URI.

The Hardhat configuration conditionally adds the Sepolia network only when both INFURA_PROJECT_ID and DEPLOYER_PRIVATE_KEY are present and valid (the private key must be exactly 64 hex characters). If either is absent, Hardhat starts with only the local HardHat network. This prevents accidental Sepolia deployments when running in a development environment without valid credentials.

The deployment script (backend/scripts/deploy.js) deploys all three contracts in sequence, waits for each deployment to be mined, and writes the resulting addresses to backend/deployed-addresses.json. The backend

configuration reads contract addresses from environment variables that must be set to the values in this JSON file after deployment. The script also prints the Etherscan URLs for each deployed contract to facilitate verification.

The get_pubkey.py utility script derives the Ethereum wallet address and public key from the deployer private key. This is used during initial setup to populate the DID for the deployer account before the full frontend registration flow is available.

A.9 AUTOMATED TEST SUITE: BACKEND/TESTS/

The backend test suite uses pytest with pytest-asyncio for async test support and httpx AsyncClient with ASGITransport for in-process HTTP testing (no actual network calls). Three test files cover the three main security domains.

A.9.1 TEST_AUTH.PY

test_health() sends GET /health and asserts 200 status with {status: ok} in the response body. This is the simplest possible integration test confirming the application starts and the routing is configured correctly.

test_login_unknown_did() sends POST /auth/login with a fabricated DID, a fake challenge hex, a fake signature hex, and a zero TOTP code. It asserts 401 status. This test confirms the DID lookup step rejects unregistered identities without raising an unhandled exception. The test uses the ASGITransport which starts the FastAPI application in-process, giving it access to the configured test database.

A.9.2 TEST_RBAC.PY

test_audit_logs_requires_auth() sends GET /audit/logs with no token and asserts the response is 401 or 403. This validates that the HTTPBearer dependency correctly blocks unauthenticated requests.

test_verify_chain_requires_admin() sends GET /audit/verify with no token and asserts 401 or 403. This validates the admin-only RBAC on the chain verification endpoint.

test_send_message_requires_auth() sends POST /messages/send with a valid-looking body but no token and asserts 401 or 403. This validates that the message relay requires authentication.

A.9.3 TEST_AUDIT.PY

This is the most security-meaningful test file because it tests the HMAC chain properties directly.

test_hmac_deterministic() calls _compute_hmac() twice with identical inputs and asserts the outputs are equal and exactly 64 characters long. This confirms the HMAC function is pure (same inputs always produce the same output) and produces the correct hex digest length for SHA-256.

test_hmac_changes_with_timestamp() calls _compute_hmac() with two different timestamps and asserts the outputs differ. This confirms the timestamp is included in the HMAC computation and that changing any input changes the output.

test_chain_breaks_on_tamper() computes a valid two-entry chain (genesis entry then a derived entry), then computes an alternative derived entry using a tampered prev_hmac string, and asserts the valid and tampered derived entries differ. This directly tests the chain-breaking property: if any entry's HMAC is changed, all subsequent entries computed against it will produce different values, which will not match the stored values computed against the original.

REFERENCES

[1] W3C, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022. https://www.w3.org/TR/did-core/

[2] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[3] D. M'Raihi et al., "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, May 2011.

[4] D. J. Bernstein, T. Lange, P. Schwabe, "The security impact of a new cryptographic library," LATINCRYPT 2012, pp. 159-176.

[5] OWASP, "OWASP Top 10 - 2021," 2021. https://owasp.org/Top10/

[6] NIST, "CVE-2024-33664," NVD, 2024. https://nvd.nist.gov/vuln/detail/CVE-2024-33664

[7] NIST, "CVE-2024-33663," NVD, 2024. https://nvd.nist.gov/vuln/detail/CVE-2024-33663

[8] Verizon, "2024 Data Breach Investigations Report," 2024. https://www.verizon.com/business/resources/reports/dbir/

[9] S. Ramirez, "FastAPI," 2024. https://fastapi.tiangolo.com/

[10] H. Lin, "Authlib," 2024. https://authlib.org/

[11] T. Lodderstedt et al., "OAuth 2.0 Security Best Current Practice," IETF RFC 9700, Jan. 2025.

[12] PyCQA, "Pylint static analysis tool for Python," 2024. https://pylint.readthedocs.io/

[13] PyCQA, "Bandit: a tool designed to find common security issues in Python code," 2024. https://bandit.readthedocs.io/

[14] Ethereum Foundation, "Solidity Documentation v0.8.24," 2024. https://docs.soliditylang.org/en/v0.8.24/

[15] M. Bayer, "SQLAlchemy 2.0 Documentation," 2024. https://docs.sqlalchemy.org/en/20/

[16] D. J. Bernstein, "TweetNaCl: a crypto library in 100 tweets," 2014. https://tweetnacl.cr.yp.to/

[17] Ethereum Foundation, "Sepolia Testnet," 2023. https://sepolia.dev/